PROGRAMACIÓN DIDÁCTICA / DE AULA: ENTORNOS DE DESARROLLO

Resumen de Unidades de Programación (Curso 2025-2026)

Código	Título de la UP	Duración	Temporalización	RA asociados (Descripción breve)
UP01	**Sistemas de control de versiones**	**15 h**	10/09/2025 - 08/10/2025	**RA04:** Control de versiones y repositorios remo-tos/colaborativos (parcial).
UP02	**Desarrollo de software**	**12 h**	08/10/2025 - 05/11/2025	**RA01:** Elementos, herramientas, fases de desarrollo y metodologías ágiles.
UP03	**Instalación y uso de IDEs**	**10 h**	05/11/2025 - 03/12/2025	**RA02:** Instalación, configuración, módulos, personalización y generación de ejecutables.
UP04	**Diseño y realización de pruebas**	**24 h**	03/12/2025 - 11/02/2026	**RA03:** Tipos de pruebas, casos de prueba, depuración, unitarias, automatización y dobles.

Código	Título de la UP	Duración	Temporalización	RA asociados (Descripción breve)
UP05	**Optimización y Documentación**	**15 h**	11/02/2026 - 25/03/2026	**RA04:** Patrones de refactorización, analizadores de código, documentación de clases e IC (parcial).
UP06	**Diagramas y modelado**	**24 h**	25/03/2026 - 03/06/2026	**RA05 y RA06:** Diagramas de clases (Estructura) y Diagramas de comportamiento (UML).

- La suma total de estas unidades es de **100 horas** (incluyendo las 10.4 horas asignadas al periodo Dual/Formación en Empresa).

UP01: SISTEMAS DE CONTROL DE VERSIONES

1. Identificación

Campo	Detalle
Código	UP01
Módulo	Entornos de Desarrollo (0487)
Duración	**15 Horas**
Temporalización	Del **10/09/2025** al **08/10/2025** (1º Trimestre)

Campo	Detalle
Bloques de Contenido	Introducción a Git; Flujos de trabajo; Gestión de repositorios locales y remotos (GitHub/GitLab)

2. Fundamentación

Resultados de Aprendizaje * **RA04 (Parcial).** Optimiza código empleando las herramientas disponibles en el entorno de desarrollo (enfocado en control de versiones local, remoto y colaborativo).

Criterios de Evaluación * **f)** Se ha realizado el control de versiones integrado en el entorno de desarrollo. * **h)** Se han utilizado repositorios remotos para el desarrollo de código colaborativo.

3. Organización

Contenidos * Introducción al control de versiones: centralizado vs. distribuido. * Instalación y configuración inicial de Git de forma local. * Ciclo de vida de los archivos en Git (Working Directory, Staging Area, Local Repository). * Gestión de ramas (Branches): creación, fusión (Merge) y resolución de conflictos básicos. * Introducción a plataformas de alojamiento remoto (GitHub / GitLab). * Operaciones distribuidas: clonación, sincronización (`push`, `pull`, `fetch`). * Trabajo colaborativo básico: Pull Requests / Merge Requests y flujos de ramificación básicos.

Metodología * Se aplican metodologías activas combinando explicaciones prácticas por comandos / interfaz gráfica e inmediatamente asimiladas mediante desafíos guiados en la consola. Se introduce el aprendizaje cooperativo simulando un entorno de desarrollo real donde el alumnado trabaja en un repositorio compartido creando ramas paralelas.

Secuencia de actividades * **A1:** Configuración de entorno local Git y simulación del ciclo de vida de archivos con commits atómicos. * **A2:** Taller de ramificación guiada: creación de características paralelas y resolución activa de conflictos de mezcla. * **A3:** Práctica de sincronización remota con GitHub mediante claves SSH/Tokens de acceso seguro. * **A4:** Simulación de proyecto colaborativo en equipos: desarrollo cooperativo usando bifurcaciones, revisiones y Pull Requests.

Recursos * Equipos del aula de informática, Git CLI, GitKraken o extensiones de VS Code, cuentas en GitHub, Aula Virtual (Aules), documentación oficial de Git.

4. Evaluación y adaptación

Instrumentos de evaluación * **Actividades prácticas y tareas en repositorio (40%):** Revisión de la estructura de commits, historial limpio y resolución de conflictos en Git. * **Cuestionarios y pruebas objetivas (60%):** Test conceptual sobre teoría del control de versiones distribuido y comandos clave.

Adaptaciones * **Medidas según necesidades:** Flexibilización de tiempos de entrega en las actividades de resolución de conflictos. * **DUA:** Uso de cheatsheets visuales de comandos Git, videotutoriales secuenciados de apoyo y opciones de comandos vs interfaces visuales alternativas.

UP02: DESARROLLO DE SOFTWARE

1. Identificación

Campo	Detalle
Código	UP02
Módulo	Entornos de Desarrollo (0487)
Duración	**12 Horas**
Temporalización	Del **08/10/2025** al **05/11/2025** (1º Trimestre)
Bloques de Contenido	Fases del ciclo de vida del software; Conceptos de compilación; Metodologías Ágiles

2. Fundamentación

Resultados de Aprendizaje * **RA01.** Reconoce los elementos y herramientas que intervienen en el desarrollo de un programa informático, analizando sus características y las fases en las que actúan hasta llegar a su puesta en funcionamiento.

Criterios de Evaluación * **a)** Se ha reconocido la relación de los programas con los componentes del sistema informático (memoria, procesador, periféricos). * **b)** Se han identificado las fases de desarrollo de una aplicación informática. * **c)** Se han diferenciado los conceptos de código fuente, objeto y ejecutable. * **d)** Se han reconocido las características de la generación de código intermedio para su ejecución en máquinas virtuales. * **e)** Se han clasificado los lenguajes de programación, identificando sus características. * **f)** Se ha evaluado la funcionalidad ofrecida por las herramientas utilizadas

en el desarrollo de software. * **g)** Se han identificado las características y escenarios de uso de las metodologías ágiles.

3. Organización

Contenidos * Arquitectura básica y ejecución de programas en el sistema (Memoria RAM, CPU). * Fases del ciclo de vida del desarrollo de software (Análisis, Diseño, Codificación, Pruebas, Despliegue, Mantenimiento). * Traductores del lenguaje: Compiladores, intérpretes y el modelo híbrido de código intermedio (Bytecode/IL) en Máquinas Virtuales (JVM, .NET). * Clasificación de paradigmas y lenguajes de programación. * El ecosistema de herramientas: editores, linters, preprocesadores, enlazadores y empaquetadores. * Marcos de trabajo ágiles: Introducción a la filosofía Agile, Scrum y Kanban.

Metodología * Exposición conceptual apoyada en mapas de arquitectura de sistemas y flujogramas de desarrollo. Se aplica la metodología de Análisis de Casos analizando fallos históricos de software por malas fases de diseño, junto con dinámicas activas de simulación ágil (tableros Kanban físicos o digitales).

Secuencia de actividades * **A1:** Mapeo interactivo de la ejecución de un código desde el editor de texto hasta los registros de la CPU. * **A2:** Simulación por grupos de un Ciclo de Vida del Software aplicando el modelo en cascada tradicional frente a un enfoque iterativo. * **A3:** Laboratorio comparativo: compilación de un fichero fuente nativo (C++/Rust) frente a uno basado en máquina virtual (Java/C#) identificando artefactos generados. * **A4:** Configuración y gestión de un Tablero Kanban para organizar el desarrollo simulado de una aplicación informática escolar.

4. Evaluación y adaptación

Instrumentos de evaluación * **Trabajos de clase y mapas conceptuales (40%):** Evaluación de entregables sobre fases del software y taxonomías de lenguajes. * **Pruebas objetivas conceptuales (60%):** Cuestionario tipo test y preguntas cortas sobre compilación, bytecode y herramientas.

Adaptaciones * **DUA:** Entrega de organizadores gráficos completados a medias para andamiar el contenido técnico; alternativas de evaluación oral para la explicación de las fases de desarrollo.

UP03: INSTALACIÓN Y USO DE IDEs

1. Identificación

Campo	Detalle
Código	UP03
Módulo	Entornos de Desarrollo (0487)
Duración	**10 Horas**
Temporalización	Del **05/11/2025** al **03/12/2025** (1º Trimestre)
Bloques de Contenido	Entornos Integrados de Desarrollo (IDEs); Extensibilidad; Automatización básica

2. Fundamentación

Resultados de Aprendizaje * **RA02.** Evalúa entornos integrados de desarrollo analizando sus características para editar código fuente y generar ejecutables.

Criterios de Evaluación * **a)** Se han instalado entornos de desarrollo, propietarios y libres. * **b)** Se han añadido y eliminado módulos en el entorno de desarrollo. * **c)** Se ha personalizado y automatizado el entorno de desarrollo. * **d)** Se ha configurado el sistema de actualización del entorno de desarrollo. * **e)** Se han generado ejecutables a partir de código fuente de diferentes lenguajes en un mismo entorno de desarrollo. * **f)** Se han generado ejecutables a partir de un mismo código fuente con varios entornos de desarrollo. * **g)** Se han identificado las características comunes y específicas de diversos entornos de desarrollo.

3. Organización

Contenidos * Definición, evolución histórica y componentes de un IDE (Editor de código, automatizador de construcción, depurador). * Estudio comparativo de IDEs modernos del mercado (Eclipse, IntelliJ IDEA, NetBeans, Visual Studio, VS Code). * Proceso de instalación limpia y gestión de arquitecturas internas mediante plugins/módulos. * Personalización avanzada del entorno: esquemas de color, snippets, atajos de teclado y perfiles de desarrollo. * Infraestructura de actualizaciones de plataformas y extensiones. * Compilación cruzada y empaquetamiento multiplataforma desde entornos de desarrollo integrados.

Metodología * Talleres 100% prácticos de instalación (“Hands-on”). El alumnado asume un rol de evaluador técnico comparando la eficiencia, consumo de recursos y facilidad de uso de dos herramientas distintas frente al mismo problema de desarrollo.

Secuencia de actividades * **A1:** Instalación guiada y configuración de entornos (ej. IntelliJ IDEA y VS Code) parametrizando canales de actualización estables/beta. * **A2:** Práctica de modularización:

instalación de extensiones de soporte de lenguaje, linters de formato y gestión de dependencias de plugins. * **A3:** Personalización productiva: creación de snippets de código propios para acelerar estructuras algorítmicas comunes. * **A4:** Matriz de evaluación comparativa: compilar un mismo proyecto Java/C# en dos IDEs diferentes, evaluando velocidad, tamaño del ejecutable resultante y experiencia de usuario.

4. Evaluación y adaptación

Instrumentos de evaluación * **Prácticas de consola e instalación de entornos (40%):** Verificación de entornos correctamente configurados y optimizados. * **Cuestionarios e informes técnicos (60%):** Evaluación de la matriz comparativa entre entornos libres y propietarios.

Adaptaciones * **DUA:** Guías visuales paso a paso de instalación con capturas de pantalla de alta calidad. Configuración de perfiles de accesibilidad dentro de los IDEs (fuentes de alto contraste, lectores de pantalla o escalado de interfaz).

UP04: DISEÑO Y REALIZACIÓN DE PRUEBAS

1. Identificación

Campo	Detalle
Código	UP04
Módulo	Entornos de Desarrollo (0487)
Duración	**24 Horas**
Temporalización	Del **03/12/2025** al **11/02/2026** (2º Trimestre)
Bloques de Contenido	Caja negra y caja blanca; Depuración; Pruebas Unitarias (JUnit); Mocks

2. Fundamentación

Resultados de Aprendizaje * **RA03.** Verifica el funcionamiento de programas diseñando y realizando pruebas.

Criterios de Evaluación * **a)** Se han identificado los diferentes tipos de pruebas. * **b)** Se han definido casos de prueba. * **c)** Se han identificado las herramientas de depuración y prueba de aplicaciones ofrecidas por el entorno de desarrollo. * **d)** Se han utilizado herramientas de depuración para definir puntos de ruptura y seguimiento. * **e)** Se han utilizado las herramientas de depuración para examinar y modificar el comportamiento de un programa en tiempo de ejecución. * **f)** Se han efectuado pruebas unitarias de clases y funciones. * **g)** Se han implementado pruebas automáticas. * **h)** Se han documentado las incidencias detectadas. * **i)** Se han utilizado dobles de prueba (mocks) para aislar los componentes durante las pruebas.

3. Organización

Contenidos * Filosofía del testing: ¿Por qué falla el software? Concepto de defecto, error y fallo. * Tipos de pruebas: Funcionales, estructurales, regresión, integración, sistema. * Técnicas de diseño de pruebas: Pruebas de caja negra (particiones de equivalencia, valores límite) y caja blanca (cobertura de líneas, caminos). * Depuración de código (Debugging): Breakpoints, breakpoints condicionales, ejecución paso a paso (`step over`, `step into`, `step out`), inspección y alteración de variables en caliente. * Pruebas Unitarias con frameworks estándar (JUnit o equivalente). Aseveraciones (`Assertions`). * Automatización del proceso de testing en el IDE. * Registro, tracking y documentación técnica de bugs/incidencias. * Aislamiento de código: Introducción a los dobles de prueba, stubs y objetos simulados (Mocks).

Metodología * Aprendizaje Basado en Problemas (ABP). Se entrega al alumnado fragmentos de código con fallos lógicos ocultos que deben rastrear mediante el depurador. Posteriormente, se evoluciona hacia el diseño formal de suites de pruebas automatizadas guiadas por especificaciones de negocio.

Secuencia de actividades * **A1:** Diseño en papel de casos de prueba por valores límite para un módulo crítico de facturación comercial. * **Lab1:** Sesión intensiva de debugging: uso de breakpoints condicionales para cazar un bucle infinito y mutar variables en caliente. * **A2:** Creación de una batería completa de pruebas unitarias automatizadas con JUnit verificando métodos matemáticos y de cadenas. * **Lab2:** Implementación de dobles de prueba (Mocks) para aislar la lógica de una clase que simula llamadas a una base de datos o API externa. * **A3:** Redacción de informes formales de reporte de bugs (Bug Reports) siguiendo plantillas de la industria.

4. Evaluación y adaptación

Instrumentos de evaluación * **Laboratorios y prácticas técnicas (40%):** Calificación directa sobre la solidez de las suites JUnit entregadas y la correcta cobertura de código lograda. * **Cuestionarios y pruebas de conocimiento (60%):** Examen teórico-práctico escrito u online sobre técnicas de diseño de pruebas.

Adaptaciones * **Medidas según necesidades:** Ampliación del tiempo permitido en las pruebas prácticas de código complejas. * **DUA:** Provisión de plantillas de código base (“boilerplate”) con la infraestructura JUnit ya montada para mitigar barreras sintácticas iniciales; uso de interfaces gráficas de cobertura visual de código (zonas verdes/rojas).

UP05: OPTIMIZACIÓN Y DOCUMENTACIÓN

1. Identificación

Campo	Detalle
Código	UP05
Módulo	Entornos de Desarrollo (0487)
Duración	**15 Horas**
Temporalización	Del **11/02/2026** al **25/03/2026** (2º y 3º Trimestre)
Bloques de Contenido	Refactorización; Análisis estático de código; Javadoc; Integración Continua

2. Fundamentación

Resultados de Aprendizaje * **RA04 (Parcial).** Optimiza código empleando las herramientas disponibles en el entorno de desarrollo (enfocado en calidad, reestructuración limpia e integración de procesos).

Criterios de Evaluación * **a)** Se han identificado los patrones de refactorización más usuales. * **b)** Se han elaborado las pruebas asociadas a la refactorización. * **c)** Se ha revisado el código fuente usando un analizador de código (análisis estático). * **d)** Se han identificado las posibilidades de configuración de un analizador de código. * **e)** Se han aplicado patrones de refactorización con las herramientas que proporciona el entorno de desarrollo. * **g)** Se han utilizado herramientas del entorno de desarrollo para documentar las clases. * **i)** Se han utilizado herramientas para la integración continua (IC) del código.

3. Organización

Contenidos * Concepto de “Código Limpio” (Clean Code) y Deuda Técnica. Olores de código (Code Smells). * Patrones de refactorización: renombrado, extracción de métodos, encapsulación de campos, eliminación de código duplicado. * El rol vital de los tests regresivos durante la refactorización. * Análisis estático de código mediante herramientas (ej. SonarQube, Linters, Checkstyle). Configuración de reglas y umbrales de calidad (Quality Gates). * Documentación técnica automatizada dentro del código fuente utilizando etiquetas estructuradas (Javadoc / Doxygen). Generación de artefactos de ayuda HTML. * Conceptos básicos de Integración Continua (CI): automatización de la compilación y pruebas en cada subida de código (GitHub Actions, GitLab CI).

Metodología * Metodología de “Refactoring Kata” (ejercicios repetitivos orientados a limpiar código intencionadamente mal escrito). El alumnado trabaja sobre proyectos reales “sucios” para dejarlos en condiciones óptimas profesionales utilizando automatismos del IDE.

Secuencia de actividades * **A1:** Detección visual y teórica de “Code Smells” en un código espagueti legado. * **Lab1:** Refactorización segura guiada por herramientas del IDE (extraer clase, extraer interfaz, renombrar variables de forma global) comprobando que los tests JUnit siguen en verde. * **A2:** Configuración de un linter estático en el IDE personalizando las reglas de estilo obligatorias del equipo de desarrollo. * **Lab2:** Documentación completa de una API orientada a objetos mediante etiquetas `@param`, `@return`, `@throws` y exportación de la documentación técnica estructurada en HTML. * **A3:** Configuración de un workflow básico de GitHub Actions que compile el código de forma automática en la nube ante cada push del alumno.

4. Evaluación y adaptación

Instrumentos de evaluación * **Prácticas de optimización de código en laboratorios (40%):** Valoración de la legibilidad, mantenibilidad y ausencia de alertas en el código refactorizado y analizado. * **Pruebas de conocimientos y test conceptuales (60%):** Cuestionarios sobre patrones de refactorización y sintaxis de documentación.

Adaptaciones * **DUA:** Uso de infografías resumen de los principales “olores de código” y sus soluciones; andamiaje de scripts de configuración YAML para la integración continua ya semi-estructurados por el docente.

UP06: DIAGRAMAS Y MODELADO

1. Identificación

Campo	Detalle
Código	UP06
Módulo	Entornos de Desarrollo (0487)
Duración	**24 Horas**
Temporalización	Del **25/03/2026** al **03/06/2026** (3º Trimestre)
Bloques de Contenido	Lenguaje Unificado de Modelado (UML); Diagramas Estructurales y de Comportamiento

2. Fundamentación

Resultados de Aprendizaje * **RA05.** Genera diagramas de clases valorando su importancia en el desarrollo de aplicaciones y empleando herramientas específicas. * **RA06.** Genera diagramas de comportamiento valorando su importancia en el desarrollo de aplicaciones y empleando herramientas específicas.

Criterios de Evaluación * **RA05:** a), b), c), d), e), f) completos (conceptos de POO, uso de herramientas, interpretación de diagramas, trazado, generación de código e ingeniería inversa). * **RA06:** a), b), c), d), e), f), g), h) completos (diagramas de comportamiento, casos de uso, interacción/secuencia, actividades, estados e interpretación/elaboración).

3. Organización

Contenidos * Introducción al modelado de sistemas informáticos: El estándar UML (Unified Modeling Language). * Diagramas Estructurales: El Diagrama de Clases (clases, atributos, métodos, visibilidad, relaciones de asociación, agregación, composición, herencia). * Ingeniería directa (generar clases de código desde el diagrama visual) e Ingeniería inversa (generar el diagrama visual analizando ficheros de código existentes). * Diagramas de Comportamiento: * Diagrama de Casos de Uso (actores, casos, relaciones `include` y `extend`). * Diagramas de Interacción/Secuencia (líneas de vida, mensajes síncronos/asíncronos). * Diagrama de Actividades (nodos de decisión, bifurcaciones, flujos de control). * Diagrama de Estados (estados, transiciones, eventos).

Metodología * Se equilibra el diseño visual analógico (pizarra/papel) con el uso de herramientas CASE de modelado modernas (ej. StarUML, PlantUML, Draw.io). Se parte del análisis de textos de requisitos comerciales donde el alumnado abstrae el dominio del problema para plasmarlo visualmente antes de programar de forma directa.

Secuencia de actividades * **A1:** Lectura de un pliego de condiciones de una aplicación y diseño del diagrama de clases identificando relaciones de herencia y agregación de componentes. * **Lab1:** Uso de herramientas de ingeniería directa para convertir un diseño UML de clases en un esqueleto de código fuente funcional en lenguaje Java o C#. * **A2:** Aplicación de ingeniería inversa sobre una biblioteca de software libre compleja para comprender visualmente su diseño de arquitectura interna de clases. * **A3:** Modelado del flujo funcional del usuario mediante diagramas de Casos de Uso y diagramas de Actividades de una aplicación de e-commerce. * **A4:** Taller práctico: diseño de un diagrama de secuencia detallado de una pasarela de autenticación segura y un diagrama de estados del ciclo de vida de un pedido online.

4. Evaluación y adaptación

Instrumentos de evaluación * **Diseño y entregas de modelado técnico (40%):** Evaluación de la exactitud semántica de los diagramas UML conforme a los estándares de la industria. * **Cuestionarios teóricos y pruebas objetivas (60%):** Exámenes dedicados a comprobar la comprensión lectora e interpretativa de planos UML estructurados.

Adaptaciones * **Medidas según necesidades:** Flexibilidad organizativa, permitiendo defensas orales grupales de los diagramas de arquitectura para justificar las decisiones de diseño tomadas. * **DUA:** Fomento del uso de PlantUML (modelado mediante código secuencial en modo texto) como una excelente alternativa accesible para alumnado con dificultades psicomotrices o de diseño puramente visual, proporcionando múltiples medios de expresión técnica.